

Báo cáo bài tập nhóm

Thành viên nhóm : Lê Anh Khoa 24022367 – Nguyễn Duy Hưng 24022349

1. Mô tả bài toán

Bài toán cờ Caro với :

- Kích thước bàn cờ 15 x 15.
- Người chơi sẽ đi trước với X và máy sẽ đi sau với O .
- Người thắng là người có 4 quân liên tiếp theo hàng ngang, hàng dọc hoặc đường chéo.
- Không xét luật chặn hai đầu
- Kết quả Hòa sẽ xảy ra khi bàn cờ đã đầy mà chưa có người thắng

2. Cách biểu diễn trạng thái bàn cờ

Bàn cờ được lưu trữ trong đối tượng `game.grid`, là một ma trận hai chiều (mảng lồng mảng) với kích thước ROWS x COLS

- **Các trạng thái ô cờ:** Mỗi ô (r, c) trong lưới nhận một trong ba giá trị định nghĩa sẵn trong `constants.py`:
 - **EMPTY**: Ô trống, chưa có quân cờ.
 - **HUMAN**: Đại diện cho quân X (người chơi).
 - **AI** : Đại diện cho quân O (máy)
- **Các trạng thái hỗ trợ**
 - **`game.last_move`**: Lưu tọa độ nước đi cuối cùng để đánh dấu trực quan trên bàn cờ.
 - **`game.win_cells`**: Một danh sách chứa tọa độ của các quân cờ tạo thành chuỗi thắng, phục vụ việc highlight khi kết thúc ván đấu.

Hàm `draw_board` chịu trách nhiệm chuyển đổi ma trận dữ liệu sang giao diện người dùng theo các thành phần:

- **Hệ thống tọa độ** : Sử dụng các biến `MARGIN` và `CELL` để tính toán vị trí thực tế trên màn hình
- Gán nhãn cột bằng chữ cái (A–N) và nhãn hàng bằng số (1–14), giúp người dùng dễ dàng định vị tọa độ tương tự như bàn cờ quốc tế.

- **Quân X (Người chơi):** Được vẽ bằng hai đường chéo cắt nhau (`pygame.draw.line`) với độ dày và khoảng đệm (`pad`) được tính toán tỉ lệ theo kích thước ô.
- **Quân O** được biểu diễn bằng một hình tròn (`pygame.draw.circle`) không đặc tâm.

3. Cách sinh nước đi hợp lệ

Hệ thống sử dụng hằng số `EMPTY` (được định nghĩa trong `constants.py`) để biểu diễn một ô cờ chưa bị chiếm đóng. Một nước đi được coi là hợp lệ khi và chỉ khi tọa độ (r, c) thỏa mãn:

- Nằm trong phạm vi bàn cờ
- `grid[r][c] == EMPTY` (Ô cờ chưa có quân của người chơi hay AI).

4. Cách kiểm tra trạng thái kết thúc

Hàm `check_win` đóng vai trò xác định xem một nước đi có dẫn đến trạng thái kết thúc ván đấu hay không:

- **Quét 4 hướng:** Ngang $(0, 1)$, Dọc $(1, 0)$, Chéo xuôi $(1, 1)$, và Chéo ngược $(1, -1)$.
- **Kiểm tra liên tục:** Tại mỗi vị trí, hệ thống kiểm tra chuỗi WIN (là 4) quân cờ cùng loại. Nếu đạt đủ số lượng, danh sách tọa độ thắng cuộc (`win_cells`) sẽ được trả về, và trạng thái `game_over` sẽ được kích hoạt, ngăn chặn việc sinh thêm các nước đi hợp lệ mới.

5. Thuật toán Minimax đã cài đặt.

I. Cơ chế hoạt động của thuật toán

- Hàm `minimax(grid, depth, is_max)` thực hiện tìm kiếm đệ quy với các bước:
 - **Trạng thái dừng:** Thuật toán dừng lại khi một bên thắng (`check_win`), bàn cờ đầy (`is_full`) hoặc đạt tới giới hạn độ sâu (`depth == 0`).
 - **Điểm thưởng (Scoring):**
 - Thắng: $+100,000 + \text{depth}$
 - Thua: $-100,000 - \text{depth}$
 - Hòa/Hết độ sâu: Sử dụng hàm `evaluate(grid)` để đánh giá điểm thế trận.
- Một số kỹ thuật tối ưu hóa
 - Bảng chuyển đổi
 - Chiến lược sinh nước đi tiềm năng
 - Sắp xếp nước đi

6. Thuật toán Alpha-Beta đã cài đặt.

I. Cơ chế Cắt tỉa Alpha-Beta

- Cải tiến từ Minimax nhằm loại bỏ các nhánh con không cần thiết trong cây quyết định

Alpha : Giá trị tốt nhất mà AI (Maximizing player) chắc chắn có thể đạt được.

Beta : Giá trị tốt nhất mà Người chơi (Minimizing player) có thể ép AI phải nhận

Điều kiện cắt tỉa: Khi `best_score >= beta` (ở lượt Max) hoặc `best_score <= alpha` (ở lượt Min), thuật toán sẽ dừng duyệt các nhánh còn lại (`break`) giúp giảm đáng kể số lượng trạng thái cần tính toán mà không làm thay đổi kết quả cuối cùng.

7. Hàm đánh giá trạng thái

- Hàm đánh giá hoạt động dựa trên việc nhận diện các mẫu hình quân cờ (patterns) theo 4 hướng: Ngang, Dọc, Chéo xuôi và Chéo ngược. Giá trị của một thế cờ được tính theo tổng điểm của AI trừ đi tổng điểm của Người chơi.
- Điểm phạt cho Người chơi cao gấp đôi điểm thưởng của AI. Điều này tạo ra một hệ thống Phòng thủ chủ động, AI sẽ ưu tiên chặn đối phương ngay khi họ có dấu hiệu hình thành chuỗi 2 hoặc 3 quân thay vì mãi mê tấn công.

8. Thiết kế các trạng thái thử nghiệm

- Trạng thái máy có thể thắng ngay
- Người chơi sắp thắng và máy cần chặn
- Đầu ván(nước đầu tiên mà máy đi)
- Trạng thái có nhiều nước đi hợp lệ khiến thuật toán phải xét nhiều nhánh
- Trạng thái giữa ván

9. Bảng kết quả thực nghiệm

STT	Trạng thái	Thuật toán	Độ sâu	Số node mà thuật toán xét	Thời gian (ms)
1	Máy Có thể thắng ngay	Minimax	4	0	0
2	Máy Có thể thắng ngay	Alpha-beta pruning	4	0	0
3	Người chơi sắp thắng và máy cần chặn	Minimax	4	0	0
4	Người chơi sắp thắng và máy cần chặn	Alpha-beta pruning	4	0	0
5	Đầu ván(nước đầu tiên mà máy đi)	Minimax	4	137.184	5590
6	Đầu ván(nước đầu tiên mà máy đi)	Alpha-beta pruning	4	3369	249
7	Đầu ván(nước đầu tiên mà máy đi)	Minimax	1	21	0
8	Đầu ván(nước đầu tiên mà máy đi)	Alpha-beta pruning	1	21	1
9	Trạng thái có nhiều nước đi hợp lệ khiến thuật toán phải xét nhiều nhánh	Alpha-beta pruning	4	3792	425
10	Trạng thái có nhiều nước đi hợp lệ khiến thuật toán phải xét nhiều nhánh	Minimax	4	132.084	9014
11	Trạng thái giữa ván	Alpha-beta pruning	4	1962	279
12	Trạng thái giữa ván	Minimax	4	122.844	7025

10. Nhận xét về số trạng thái đã xét và thời gian chạy

I .So sánh trực tiếp hai thuật toán ở cùng 1 độ sâu

- 2 thuật toán chọn cùng nước đi với nhau
- **Hiệu quả cắt nhánh của Alpha-Beta pruning** : Thuật toán Alpha-Beta pruning giảm thiểu đáng kể số lượng trạng thái phải xét. Ví dụ, ở trạng thái "Nhiều nước đi hợp lệ", Minimax phải duyệt **132.084** nodes, trong khi Alpha-beta chỉ cần **3.792** nodes (giảm khoảng **35 lần**)
- **Thời gian chạy**: Tốc độ phản hồi của Alpha-beta nhanh hơn vượt trội. Tại trạng thái "Nhiều nhánh", thời gian giảm từ **9014 ms (9 giây)** xuống chỉ còn **425 ms** (chưa tới 0.5 giây).
- **Kết luận**: Alpha-beta pruning loại bỏ được các nhánh không cần thiết mà không làm thay đổi kết quả cuối cùng, giúp AI chơi mượt mà hơn trong điều kiện thực tế

II. Cơ chế Heuristic đã sử dụng

Tại các trạng thái "Máy có thể thắng ngay" hoặc "Người sắp thắng":

- **Trước khi khởi động các thuật toán tìm kiếm cây**, AI sẽ gọi hàm `find_immediate_win` để quét nhanh toàn bộ các nước đi tiềm năng.
- **Ưu tiên 1:** Nếu AI có nước đi tạo thành chuỗi 4 quân, AI thực hiện ngay để kết thúc trận đấu.
- **Ưu tiên 2:** Nếu người chơi đang có chuỗi 3 quân thoáng, AI sẽ thực hiện nước chặn (block) ngay lập tức
- **Kết quả đo đạc:** Ở các trạng thái này, hệ thống sẽ ghi nhận **0 nodes** và **0 ms** do không cần duyệt cây không gian trạng thái.

11. Sự ảnh hưởng của độ sâu tìm kiếm

- **Độ sâu 1:** Cả hai thuật toán đều xét số node cực thấp và thời gian gần như bằng 0 dù ở trạng thái nào .
- **Độ sâu 4:** Số node bùng nổ theo hàm mũ tăng từ hàng chục lên hàng chục nghìn node (dựa trên dữ liệu của trạng thái đầu ván) và thời gian chạy cũng tăng lên hàng nghìn lần .
- **Nhận xét:** Độ sâu càng lớn, thuật toán càng thông minh vì nhìn xa được nhiều nước đi hơn, nhưng gánh nặng tính toán tăng lên rất nhanh.

12. ưu điểm và nhược điểm của chương trình

- Hàm đánh giá sử dụng heuristic dựa trên số quân liên tiếp và số đầu mở để tính điểm cho bàn cờ. Ưu điểm của phương pháp này là đơn giản, dễ cài đặt, tốc độ xử lý nhanh và có khả năng phân biệt giữa trạng thái tấn công và phòng thủ.
- Tuy nhiên hàm đánh giá vẫn chưa nhận diện được các tình huống đe dọa kép , với luật chơi bên đầu tiên được 4 ô liên tiếp là chiến thắng thì máy trong vai trò đi sau sẽ gặp bất lợi cực lớn do đó chương trình chưa tìm ra cách để chiến thắng 100% .

13.Tài liệu tham khảo và phần đã tham khảo từ các repo mẫu

- Chương trình đã tham khảo ý tưởng cách ghi và lưu lại log nước đi, tổ chức và đặt tên các thư mục , và ý tưởng xây dựng hàm đánh giá trạng thái từ repo <https://github.com/husus/gomokuAI-py>